

INSTITUIÇÃO: IFF Campus Campos Centro

CURSO: Bacharelado em Engenharia de Controle e Automação

DISCIPLINA: Microcontroladores e Microprocessadores

PROJETO: Compilador de linguagem C para microcontrolador PIC18F1220

AUTOR: Roger Almeida Gomes

DATA: 20/08/2025

1 Objetivo do projeto

Este projeto foi desenvolvido na disciplina de Microcontroladores e Microprocessadores com o objetivo de entender, na prática, o caminho que um código em C percorre até virar um programa executável em um microcontrolador.

A proposta foi montar um compilador simples de C para o PIC18F1220, escrito em Python. Ele recebe um código em C, passa pelo GCC, traduz o resultado para Assembly do PIC18 e, no fim, gera um arquivo Intel HEX que pode ser gravado no chip. Para testar o fluxo completo, usei o exemplo clássico do Pisca-LED.

2 Módulos do projeto

O sistema foi dividido em quatro módulos:

- **Geração do Código Simplificado:** o código-fonte em C é compilado pelo GCC (GNU Compiler Collection), via terminal, para extrair a árvore de sintaxe simplificada no formato .gimple utilizando o comando `gcc -c -O0 -fdump-tree-gimple LED_BlinkC.c`.
- **Tradução para Assembly (parser):** um script em Python lê o arquivo .gimple e traduz as estruturas de alto nível para instruções Assembly do conjunto de instruções do PIC18F1220.
- **Geração do Executável Binário (montador):** o arquivo .asm é processado, cada instrução é convertida no respectivo opcode em hexadecimal e o resultado é escrito no formato Intel HEX 8.
- **Validação em Ambiente Virtual:** o arquivo .hex é carregado em um PIC virtual no Proteus 8, para verificar em tempo real se o comportamento dos pinos corresponde ao esperado.

3 Arquitetura do compilador e fluxo de dados

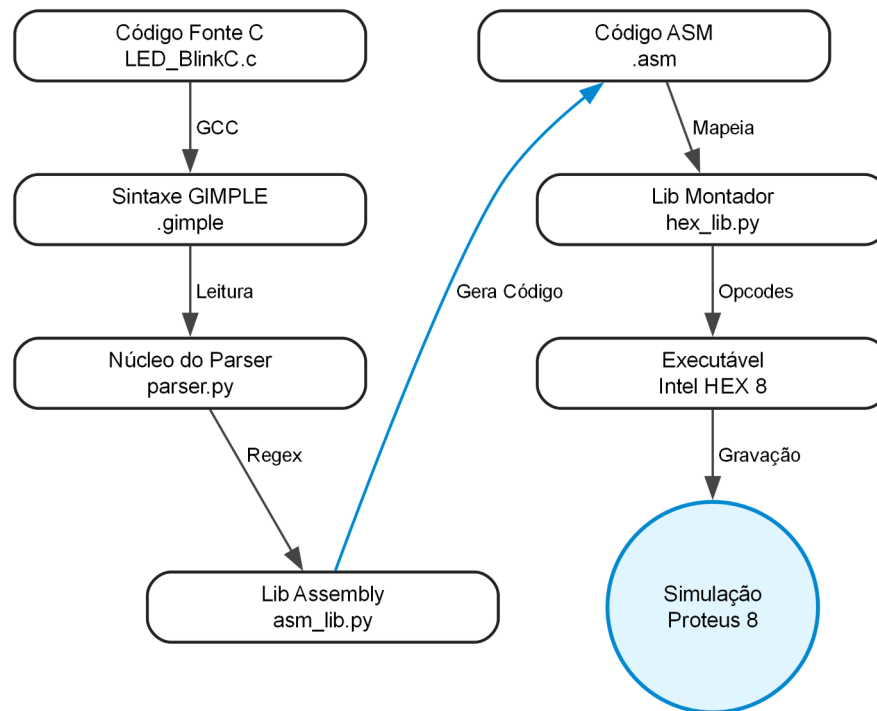


Figura 1: Fluxo de dados do compilador, do código C à simulação no Proteus 8.

3.1 Detalhamento do fluxo

- **Etapa 1, geração do Assembly (coluna da esquerda):** o código em C entra no front-end do GCC, que gera o .gimple. O script parser.py lê esse arquivo linha por linha e aciona a biblioteca asm_lib.py para mapear os registradores e mnemônicos equivalentes.
- **Link de Transição (Gera Código):** com a listagem em Assembly (.asm) gerada, os dados passam para a segunda etapa.
- **Etapa 2, montagem e validação (coluna da direita):** o arquivo .asm é a entrada da biblioteca hex_lib.py, que calcula os opcodes de 16 e 32 bits e escreve as linhas no padrão Intel HEX 8. Por fim, o firmware é carregado na memória flash do microcontrolador simulado no Proteus 8, para validação visual.

4 Desenvolvimento Técnico do Parser

O GIMPLE quebra uma instrução de C em várias linhas bem simples, então não dá para traduzir linha por linha de forma isolada: às vezes é preciso olhar várias linhas seguidas para entender o que o programa quer fazer. Por isso o parser combina expressões regulares com uma pequena máquina de estados.

No código, variáveis de estado (`estado_sequencia_at`, `estado_sequencia_inc` e `estado_sequencia_cond`) indicam em que ponto de uma sequência o parser está, seja uma atribuição, um incremento ou uma condição. Quando uma sequência se completa, o parser escreve as instruções correspondentes no arquivo `.asm`.

5 Geração de Código Assembly

O PIC18 trabalha com 8 bits, então variáveis maiores precisam ser divididas em bytes. A `asm_lib.py` cuida disso da seguinte forma:

- **Gerenciamento de Tipos:** variáveis do tipo `int` (16 bits) ocupam 2 bytes consecutivos na SRAM, enquanto as do tipo `long int` (32 bits) ocupam 4. Os endereços são definidos com `EQU`.
- **Incremento com mais de um byte:** depois de incrementar o byte menos significativo, o bit Z do registrador STATUS é testado com `BTFSC STATUS, Z`, `ACCESS`. Se o byte voltou a zero, houve estouro e o incremento continua no byte seguinte.
- **Comparações Lógicas:** para comparar variáveis de vários bytes com um valor constante, cada byte é comparado com o byte com o byte correspondente da constante usando `XORLW`, do mais significativo para o menos significativo. A execução desvia com `BNZ` assim que algum byte difere, evitando comparar o restante desnecessariamente.

6 O Montador e a decodificação de opcodes

A biblioteca `hex_lib.py` converte os mnemônicos gerados em opcodes hexadecimais de 16 bits, seguindo o *datasheet* do PIC18F1220.

- **Instruções de Palavra Simples (16 bits):** para instruções de manipulação direta como MOVLW, MOVWF, MOVF e XORLW, o opcode é obtido combinando os bits fixos da instrução com o endereço de 8 bits do registrador, usando máscaras e OR bit a bit. O bit de acesso (a) altera o opcode. No MOVWF, por exemplo, o prefixo é 0x6E para Access Bank e 0x6F para banked.
- **Instruções de Palavra Dupla (32 bits):** em instruções de salto como GOTO e CALL, o endereço de destino não cabe em 16 bits, então a instrução ocupa duas palavras. Sendo K o endereço de destino em palavras (endereço em bytes dividido por 2), a primeira palavra recebe os 8 bits inferiores (K & 0xFF) somados à base 0xEF00 (GOTO) ou 0xEC00 (CALL). A segunda recebe os bits restantes (K >> 8 & 0xFFFF) somados a 0xF000.

7 Resultados e Validação no Hardware Simulado

Para validar o compilador, o arquivo final no formato Intel HEX 8 gerado pelo script foi carregado em uma simulação no software Proteus 8.

- **Cenário de Teste:** circuito com o PIC18F1220, com os pinos de oscilação e reset (MCLR) configurados e LEDs conectados às saídas digitais RB0 e RB1.
- **Execução:** o arquivo .hex gerado pelo compilador em Python foi carregado na memória flash do PIC18 virtual e a simulação foi iniciada.
- **Resultado:** os LEDs alternaram como esperado, com o delay funcionando. Isso mostra que o parser, a divisão das variáveis na asm_lib e a geração de opcodes na hex_lib funcionam em conjunto para esse programa.

8 Materiais complementares

O código-fonte do compilador, o exemplo em C usado no teste, o arquivo .hex gerado e o vídeo da simulação estão disponíveis no repositório do portfólio:

<https://github.com/rogeralmeida-11/portfolio-academico/blob/main/README.md>